10/22: Lecture 17. L VS. NL

1 Warm-Up

Problem. Design a (Nondeterministic?) Log-Space algorithm for:

- Π, given $\Pi = L(N)$ for some NFA N (i.e. there is some NFA N that recognizes Π).
  (Deterministic). Recall that NFAs recognize regular languages, and DFAs are DTMs that use $O(1)$ space, since they only remember the state that it is currently in. Since NFAs are equivalent to DFAs, simply (1) Convert the NFA to a DFA, then (2) Simulate the DFA.
- $\{0^k 1^k : k \in \mathbb{N}\}$.
  (Deterministic). We can maintain a counter that we increment for each 0 and decrement for each 1. We reject if a 0 appears after a 1, or if the counter becomes negative. At the end, accept only if the counter is 0. Since the strength has length n, the counter only requires $O(\log n)$ bits/space.
- BalancedParentheses.
  (Deterministic). We can maintain a counter that we increment for each "(" and decrement for each ")". We reject if the counter ever becomes negative. At the end, accept only if the counter is 0. Again, the counter requires $O(\log n)$ space.
- Directed st-Connectivity.
  (Nondeterministic). Instead of storing the entire path, we can use a nondeterministic algorithm that guesses one vertex at a time. We start at s, nondeterministically guess the next vertex, verify that a directed edge exists to that vertex, and repeat until reaching t, or reject the branch after at most $n-1$ edges. Storing the current vertex costs $O(\log n)$, storing the guessed next vertex costs $O(\log n)$, and storing the total number of steps taken costs $O(\log n)$ (i.e. this problem is in NL).
- Undirected st-Connectivity.
  (Nondeterministic). This is a special case of Directed st-Connectivity, so we can use the same nondeterministic path-guessing algorithm as the directed problem (i.e. this problem is in NL).
  (Deterministic). It turns out that this problem is also in L! (But it was an open problem for a long time).

2 L VS. NL

2.1 Review: What We Know So Far

We know that by definition,

$$\mathrm{L} \subseteq \mathrm{NL}.$$

By Savitch's Theorem,

$$\mathrm{L} \subseteq \mathrm{NL} \subseteq \mathrm{SPACE}[(\log n)^2].$$

Finally, by the Space-Hierarchy Theorem:

$$\mathrm{L} \subsetneq \mathrm{SPACE}[(\log n)^2].$$

This means that there are problems solvable in $O(\log^2 n)$ space that cannot be solved in $O(\log n)$ space.

However, it remains open whether L = NL (though we believe strongly that they are not equal, much as how we believe P $\neq$ NP).

2.2 Using Mapping Reductions

Motivation. We want to define NL-hard and NL-complete problems, just as we previously defined NP-hard and NP-complete problems. Ultimately, our goal is to find problems s.t.

$$\text{NL-complete problem} \in \text{L} \Rightarrow \text{L} = \text{NL}.$$

Reductions. What kind of reductions can give us NL-completeness?

- We can't use poly-time reductions, because $\text{L} \subseteq \text{NL} \subseteq \text{P}$.
- For instance, even if a problem $B \in \text{L}$, a poly-time reduction from another problem A to B (i.e. showing $A \leq_{\text{P}} B$ could require much more than $O(\log n)$ space, which would not prove that $A \in \text{L}$.

3 Log-Space Mapping Reductions

Definition. A problem A *log-space mapping reduces* to another problem B, i.e. $A \leq_{\text{L}} B$, if there is a function f that satisfies the following conditions:

1. (Mapping Reduction Definition) $a \in A \Leftrightarrow f(a) \in B$.
2. (Log-Space Definition) f can be computed using $O(\log n)$ total work space (where $n = |a|$).

3.1 Log-Space Transducers

A TM R is a *Log-Space Transducer* that computes the reduction f if it has the following properties:

1. R has a read-only input tape
2. R uses at most $c \cdot \log n$ total work space
3. R has a *write-once* output tape (i.e. the tape head on the output tape cannot transition left).

Property 3. The output tape is write-once because the reduction may produce an output much larger than $O(\log n)$, so we can not use it as extra working memory.

3.2 Properties of Log-Space Mapping Reductions

Transitivity. $A \leq_{\text{L}} B$ and $B \leq_{\text{L}} C \Rightarrow A \leq_{\text{L}} C$.

Effective Completeness. $A \leq_{\text{L}} B$ and $B \in \text{L} \Rightarrow A \in \text{L}$. (also: $A \notin \text{L}$ and $A \leq_{\text{L}} B \Rightarrow B \notin \text{L}$).

- **Naive Proof Attempt.** Suppose some $B \in \text{L}$, and let M is a log-space machine that decides B. Next, for some $a \in A$, let R be a log-space transducer computing the reduction $f(a)$. We propose the following approach:

  1. Run R to compute $f(a)$.
  2. Store $f(a)$.
  3. Run M on $f(a)$.

  The problem is that in order to run M, we need to give it $f(a)$ as input. However, $f(x)$ could be poly-size, so we cannot store the entire input using only $O(\log n)$ space.

- **Proof.** (Instead of storing information, recompute).
  Rather than storing $f(a)$, we recompute individual characters of the input whenever M needs them:

  1. Keep track of where M is reading: we store an index i, representing the position of M's input tape head in $f(a)$.
  2. When M wants to read character $f(x)_i$, rerun R from the beginning on the original input x.
  3. Count the characters produced by R: Instead of storing its output, maintain a counter that tracks how many characters R has generated.

4. When the counter reaches i, give that character to M, since we only need to remember that one character and not the entire output.
5. Continue running M: Whenever M needs another character of $f(x)$, repeat this process.

With this approach, we only need to store:

- M's working memory: $O(\log n)$
- R's working memory: $O(\log n)$
- Index i: $O(\log n)$
- The counter tracking R's output position: $O(\log n)$
- Output character $f(x)_i$: $O(1)$

3.3 NL-Hard and NL-Complete

Definition. A language B is *NL-hard* if:

$$\forall A \in \text{NL}. A \leq_{\text{L}} B,$$

i.e. every problem in NL can be reduced to B using log-space mapping reductions.

Definition. A language B is *NL-complete* if $B \in \text{NL}$ and B is NL-hard.

Theorem. DIRECTED *st*-CONNECTIVITY is NL-Complete.

Corollary. If DIRECTED *st*-CONNECTIVITY $\in$ L, then L = NL.

3.4 NL and coNL

Fact.

$$\text{NL} = \text{coNL}.$$

(Note that despite this, we do not know whether L = NL.)